# DAA College Assignment — Inventory Search System

The complete implementation of the **Inventory Search System** is located in the `inventory-search-system/` directory.

## Quick Start

**1. Start Backend:**

```
cd "DAA Assignment/inventory-search-system/backend"

npm install

npm start
```

Runs at: `http://localhost:5000`

**2. Start Frontend:**

```
cd "DAA Assignment/inventory-search-system/frontend"

npm install

npm run dev
```

Runs at: `http://localhost:5173`

---

## 3. Data Structure Comparison

Side-by-side asymptotic time and space complexity comparison:

| Operation | AVL Tree | Hash Map | Key Observation |
|---|---|---|---|
| **Search** | `O(log n)` guaranteed | `O(1)` average | Hash Map provides faster point lookup; AVL guarantees worst-case bound |
| **Insert** | `O(log n)` guaranteed | `O(1)` average | AVL maintains strict balance via rotations (LL, RR, LR, RL) |
| **Delete** | `O(log n)` guaranteed | `O(1)` average | AVL may require O(log n) rebalancing rotations up to root |

| Operation | AVL Tree | Hash Map | Key Observation |
|---|---|---|---|
| **Ordered Traversal** | `O(n)` (In-Order) | Not naturally ordered | AVL naturally outputs sorted inventory items by Product ID |
| **Space Complexity** | `O(n)` | `O(n)` | AVL stores node pointers (left/right/height); Hash Map stores buckets |

** Note: Hash Map operations are O(1) on average. In the theoretical worst-case (heavy hash collisions or resize rehashing), Hash Map operations can degrade to O(n). Conversely, the AVL Tree strictly guarantees O(log n) worst-case performance under all circumstances.*

---

## 4. Complexity Analysis

Summary of algorithmic bounds and architectural roles:

### AVL Tree (Self-Balancing BST) — Strictly Balanced

- **Search:** `O(log n)`
- **Insertion:** `O(log n)`
- **Deletion:** `O(log n)`
- **Space:** `O(n)`

*AVL Tree maintains sorted data and guarantees O(log n) search because the tree remains strictly balanced via rotations.*

### Hash Map (JavaScript Map) — Key Indexing

- **Search:** `O(1)` average
- **Insertion:** `O(1)` average
- **Deletion:** `O(1)` average
- **Space:** `O(n)`

*Hash Map provides average O(1) direct lookup using a key via hash table buckets.*

---

## 5. Why Both Data Structures?

Complementary roles in an inventory search architecture:

- **AVL Tree:** Used when we need **ordered data** and guaranteed logarithmic search, insertion and deletion (enabling range queries and sorted inventory listings).

- **Hash Map:** Used when we need **very fast direct lookup** using Product ID without any tree traversal overhead.

---

For detailed algorithm explanations, time and space complexity proofs, and architecture diagrams, see the Project README.